Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-extension-guidelines-00
Published:	14 November 2026
Intended Status:	Best Current Practice
Expires:	18 May 2027
Authors:	M. Wullink	P. Kowalik
---	---	---
	SIDN Labs	*DENIC*

Guidelines for Extending the RESTful Provisioning Protocol (RPP)

Abstract

The RESTful Provisioning Protocol (RPP) defines a base set of data objects, component objects, and operations for provisioning registry data. This document defines guidelines and a normative mechanism for extension specifications that add data elements, associations, operations, result codes, or other functionality, without modifying the base specifications itself. These guidelines specify the recognized RPP extension points, the JSON Schema composition method extensions MUST use to remain independently combinable, the corresponding registration requirements, and considerations for client and server implementations that support one or more extensions.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and

restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
2. Terminology 5
3. Conventions Used in This Document 6
4. Restrictions on Extensions 6
5. RPP Extension Points 6
   5.1. Shared Components 7
   5.2. Data Objects 7
   5.3. Data Elements 7
   5.4. Operations 7
   5.5. URL Endpoints 7
   5.6. Operation Inputs and Outputs 7
   5.7. External Data Types 8
   5.8. Result Codes 8
   5.9. Problem Details 8
   5.10. Query Parameters 8
   5.11. HTTP Headers 8
   5.12. New Authentication and Authorization Methods 8
   5.13. Discovery document 8
6. Selecting an Extension Point 9
7. Identifying Extensions 10
8. Extending Data Objects 10
   8.1. JSON Schema Composition 10
   8.2. Schema Identification 11
   8.3. Extension Dependencies 11
   8.4. Consumer Implementation 11
      8.4.1. Java 12

8.5. Registration 12
8.6. Validation 12
9. Extension Specification Format 12
10. IANA Registries for Extensions 14
11. IANA Considerations 15
12. Security Considerations 15
13. Internationalization Considerations 16
14. Privacy Considerations 16
15. Change History 16
16. References 16
16.1. Normative References 16
16.2. Informative References 17
Appendix A. Example Extension 17
A.1. Introduction 18
A.2. Extension Overview 18
A.3. Shared Components 18
A.4. Data Objects 18
A.5. Data Elements 18
A.5.1. Domain Name Data Object 18
A.6. Operations 19
A.7. URL Endpoints 19
A.8. Operation Inputs and Outputs 19
A.9. External Data Types 19
A.10. Result Codes 19
A.11. Problem Details 19
A.12. Query Parameters 19
A.13. HTTP Headers 19
A.14. Authentication and Authorization Methods 19
A.15. Discovery Document 19
A.16. JSON Schema 20

A.17. Examples 20
A.18. Composition and Validation 21
A.19. IANA Considerations 22
A.20. Security Considerations 22
A.21. Privacy Considerations 22
A.22. Internationalization Considerations 22
Acknowledgements 22
Authors' Addresses 22

1. Introduction

The RESTful Provisioning Protocol (RPP), as defined in [I-D.ietf-rpp-core], [I-D.ietf-rpp-data-objects], and [I-D.ietf-rpp-json], provides a base set of data objects, component objects, and operations for the provisioning of registry data. Deployments may need to add functionality beyond this base set, for example to support policies specific to a registry, a jurisdiction, or a class of registered objects. RPP is designed to accommodate such needs through extension specifications, so that additional functionality can be introduced without modifying the base specification that defines the object or operation being extended.

Because independently developed extension specifications may be deployed together, and because RPP relies on JSON Schema, as defined in [I-D.ietf-rpp-json], to validate protocol messages, extensions need to follow a common, predictable pattern. Without such a pattern, extensions that are individually valid could conflict when combined at a given deployment, or could require ad-hoc, per-combination schema and code to be written for every deployment profile. This document defines that common pattern.

This document identifies the set of extension points along which an RPP base specification MAY be extended, including new data objects, new data elements on existing objects, new operations, new operation inputs and outputs, external data types, new RPP result codes, error object extension fields, additional query parameters, and additions to the RPP Discovery document. For each extension point, this document states the applicable registration requirements.

This document further discusses considerations for client and server implementations that need to compose, validate, and process RPP messages that may include data contributed by more than one extension.

An extension specification MAY be published as an Internet-Draft and RFC when the extension requires coordination, interoperability and broad distribution. The intended status of such a document depends on what is extended and on the amount of general interest in the extension. An extension that is intended for a closed environment or a limited audience

does not need to be published, and MAY be documented in a form that is appropriate for its audience. Such a private extension SHOULD still follow the structure and the rules in this document, so that it can be combined with other extensions at a deployment.

This document does not itself define any RPP extension. It defines the rules that extension specifications, and the deployments that support them, are expected to follow.

TODO: Make sure there is no normative language in the sections below that should be moved to the core specifications

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [RFC3986].

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

Base specification - The RPP specification, such as [I-D.ietf-rpp-data-objects], [I-D.ietf-rpp-json] or [I-D.ietf-rpp-core], that originally defines a data object, component object, or operation.

Extension specification - A specification that adds new data elements, associations, or operations to an object or operation defined in a base specification, without modifying the base specification itself.

Data object - A top-level provisioned resource object that has an independent lifecycle and identity, and defines the operations that can be performed on it, as defined in [I-D.ietf-rpp-data-objects].

Component object - A reusable data structure that carries data only, has no operations of its own, and is embedded within data objects or other objects, as defined in [I-D.ietf-rpp-data-objects].

Process object - An object that represents a long-running or multi-step operation initiated on a data object, and whose lifecycle is bound to that data object, as defined in [I-D.ietf-rpp-data-objects].

Data element - A logical unit of information of an object, identified by a stable name and defined by its cardinality, mutability, data type and constraints, as defined in [I-D.ietf-rpp-data-objects]. In the JSON representation, a data element is represented as a property.

Transient data element - A data element that is part of the input or output of an operation but is not persisted as part of the state of an object.

Base object - A data object, component object or process object defined in a base specification, that is changed by one or more extension specifications.

Consumer - A server or client that generates, processes or validates RPP messages.

Deployment - A server together with the set of extensions it supports.

Profile - A named set of protocol features, versions and extensions that defines the capabilities of an RPP server, as defined in [I-D.ietf-rpp-core].

Discovery document - The JSON document, published by an RPP server at a well-known location, that describes the capabilities of the server, including the extensions it supports, as defined in [I-D.ietf-rpp-core].

JSON Schema - A vocabulary for describing and validating JSON documents, as defined in [JSON-SCHEMA]. This document uses JSON Schema draft 2020-12.

`rpp:extends` - The property of an extension JSON Schema that maps the reference of each base object definition to the local definition in the extension schema that contributes the new properties, as defined in [I-D.ietf-rpp-json].

Effective schema - The JSON Schema that is composed, for one base object definition, from that definition and the definitions of all extensions that a deployment supports, using the `rpp:extends` mappings of the extension schemas. The effective schema has its own `$id`. The effective schema is also referred to as the combined schema.

Combined schema - The JSON Schema produced by composing the JSON Schema definition of a base object with the JSON Schema contributed by every extension specification supported by a given deployment or profile, as defined in [I-D.ietf-rpp-core]. See effective schema.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT","SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

4. Restrictions on Extensions

An extension MUST NOT remove, redefine, or otherwise alter the semantics of a property defined by the RPP base specification. An extension MUST only add new properties or associations, and MUST ensure that any new elements do not conflict with existing ones in the base specification.

TODO: do we recommend to use namespaces, using prefixes, to avoid naming conflicts between properties added by different extensions?

5. RPP Extension Points

An extension specification MAY extend RPP along one or more of the following independent extension points. Each extension point has its own registration requirements and, where applicable, its own schema-composition method.

5.1. Shared Components

Extensions MAY define shared components, such as common data types or utility functions, that can be reused across multiple data objects, operations, or other extension points.

5.2. Data Objects

Extensions MAY define entirely new resource, component, or process objects, as described in [I-D.ietf-rpp-data-objects].

5.3. Data Elements

Extensions MAY add new data elements (properties) to an existing data object, component object, or process object defined in a base specification. This is the extension point covered in detail inSection 8 below.

5.4. Operations

Extensions MAY define entirely new operations on existing or new data objects.

5.5. URL Endpoints

RPP endpoint URLs are not an independent extension point. As defined in [I-D.ietf-rpp-core], every endpoint URL and HTTP method is derived mechanically from a Data Object's `"Identifier"`, from its operations, and from the Direct Access flag on its data elements; no endpoint is defined independently of a corresponding Data Object. Consequently:

- A new Data Object automatically obtains a collection endpoint once its `"Identifier"` is registered, by applying the `plural()` derivation rule.
- A new association with Direct Access set to `true` automatically obtains a sub-resource path, derived recursively from the parent resource's path.
- A new operation automatically obtains its HTTP method and URL path once its `"Identifier"` is registered.

An extension specification MUST NOT define ad-hoc endpoint URLs or HTTP methods. It MUST instead register the underlying data object, association, or operation as described in the corresponding extension point above, and rely on the derivation rules in [I-D.ietf-rpp-core] to determine the resulting endpoint.

5.6. Operation Inputs and Outputs

Extensions MAY add both transient and persistent data elements to the input or output of an existing operation.

5.7. External Data Types

Instead of adding properties to an object, an extension MAY reference a type defined entirely in an external specification (e.g. `External:RPP-JSContact-Profile:Card`), as described in [I-D.ietf-rpp-data-objects]. This is the appropriate mechanism when substituting or supplying a whole, independently versioned data type, rather than adding fields to an existing RPP-defined object.

5.8. Result Codes

Extensions MAY define new RPP result codes within the reserved result code classes, registered in the RPP Result Codes registry defined in [I-D.ietf-rpp-core]. New result codes MUST be registered in the RPP Result Codes registry defined in [I-D.ietf-rpp-core].

5.9. Problem Details

Extensions MAY add new fields to the Problem Detail error object to convey additional, extension-specific information about the cause of an error, as described in [I-D.ietf-rpp-core].

TODO: there is no schema for the Problem Detail error object defined in the base specification, fix this?

5.10. Query Parameters

Extensions MAY define additional HTTP query parameters for existing operations, for example to further qualify a check or read request, as described in [I-D.ietf-rpp-core]. Any transient parameter for a read, query, update, and delete operation MUST be translated into an HTTP query parameter, named by the operation's parameter identifier.

5.11. HTTP Headers

Extensions MAY define new HTTP headers for existing operations, as described in [I-D.ietf-rpp-core]. Any new headers MUST be registered in the appropriate IANA registry.

5.12. New Authentication and Authorization Methods

TODO: is this an extension point?

5.13. Discovery document

A server that supports one or more extensions MUST publish every supported extension in the `extensions` list of its RPP Discovery document, as defined in [I-D.ietf-rpp-core]. Each entry MUST contain the `name`, `id`, `version` and `url` of the extension, using the name, identifier and version defined in the Extension Overview section of the extension specification.

A client MUST NOT use an extension that is not listed in the Discovery document of the server. A server MUST reject a request that uses an extension that is not listed in its Discovery document with the RPP result code 02103 (Unimplemented extension) defined in

[I-D.ietf-rpp-core], and MUST NOT include data of an unlisted extension in its responses. A server MAY restrict the use of a listed extension to specific clients, based on the identity and the authorisations of the client. The Discovery document is available without authentication, so a client can find extensions listed that it is not authorised to use. A server MUST reject a request from an unauthorised client for such an extension with an authorisation error.

New properties added by an extension to the RPP Discovery document MUST be included in the appropriate section of the document, following the structure defined in [I-D.ietf-rpp-core].

TODO: there is no schema for the Problem Detail error object defined in the base specification, fix this?

6. Selecting an Extension Point

An extension designer MUST use the simplest extension point that fulfils the requirement. The forms below are listed in order of increasing complexity and of increasing impact on implementations. A designer SHOULD only move to the next form when the answer to the question of the current form is "no".

1. Reuse an existing extension point of a base specification. The base specifications define open-ended elements to which new values can be added without adding new properties, for example the method of the authorisation information object, the labels of status objects, the labels of labelled and dictionary associations (such as the roles of the contacts of a domain name), and the keys of dictionary data types. Question: can the requirement be met by new values of such an element?
2. Add data elements to an existing object, or add transient data elements to the input or output of an existing operation, as described in Section 8. Question: can the requirement be met by adding to an existing object or operation?
3. Define a new data object. A new data object SHOULD differ significantly from existing objects. If a new object replicates a significant number of the data elements of an existing object, the question of the previous form was probably answered incorrectly. Question: can the requirement be met by applying the uniform interface operations to a new object?
4. Define a new operation, or a new process object for a long-running or multi-step operation. Question: can the requirement be met by new operations on new or existing objects?
5. Add protocol-level elements, such as new HTTP headers, query parameters, result codes, Problem Detail fields or Discovery document properties. These elements apply independent of an object, and have the largest impact on implementations.

Before defining an extension, a designer SHOULD check the IANA RPP Extensions registry, the RPP Data Object Registry and the published extension specifications for an existing extension that provides the same function, and SHOULD reuse it. Multiple extensions that realise the same function fragment the ecosystem, and SHOULD be avoided.

7. Identifying Extensions

Every extension has an identifier, which is published in the `id` field of the extension entry in the Discovery document.

- The identifier of a standardised extension MUST be a URN below the RPP URN sub-namespace defined in [I-D.ietf-rpp-core], of the form `urn:ietf:params:rpp:extension:<name>`, and MUST be registered as described in Section 10.
- The identifier of a private extension MUST be a globally unique URI, MUST NOT use the RPP URN sub-namespace, and SHOULD be based on a domain name that is controlled by the party that defines the extension.
- The version of an extension is published in the `version` field. The identifier of an extension SHOULD NOT contain the version, so that the identifier remains stable when the extension is revised.

TODO: the examples in [I-D.ietf-rpp-core] include the version in the identifier of an extension in one place and not in another. This must be aligned.

8. Extending Data Objects

RPP is designed so that extension specifications can add new data elements to data objects, component objects, and operations without modifying the specification that originally defines them, as described in [I-D.ietf-rpp-data-objects]. This section defines the normative method an extension specification MUST follow when it adds properties to an object defined in a base specification, so that JSON Schema validation, as defined in [I-D.ietf-rpp-json], keeps functioning correctly across specifications.

8.1. JSON Schema Composition

Every data object defined in a base specification, when modified by an extension specification, must be extended using an extension specification. The extension specification MUST define a new JSON Schema document that composes the base object's schema with the new properties using `allOf`. The extension specification MUST NOT copy, redefine, or otherwise restate the JSON Schema definition of the base object. Instead, it MUST reference the base object's definition by its fully qualified `$id`-relative pointer (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) in an `allOf` branch, and add its own new properties in a separate branch.

An extension schema MUST declare which base object definitions it extends using the `rpp:extends` property, as defined in [I-D.ietf-rpp-json]. The property maps the fully qualified reference of each base object definition to the local definition, within the extension schema, that contributes the new properties. This mapping allows an implementation to construct, for a given base object, the effective schema that combines the base definition with every supported extension, without any knowledge of the extensions in advance.

8.2. Schema Identification

A specification that defines JSON Schema for RPP objects MUST assign a stable `$id` to its schema document(s). Extension specifications MUST reference a base object's definition by its `$id` (e.g. `https://www.iana.org/.../domainName.json#/$defs/domainName`) rather than copying the base definition into their own schema.

8.3. Extension Dependencies

Extensions SHOULD be independent of one another. An extension MAY however extend a definition that is introduced by another extension, for example a new data object that is defined by an extension and that is changed by a second extension. In that case the second extension MUST reference the definition in the schema of the first extension by its `$id`-relative pointer in its `rpp:extends` property, and MUST list the first extension in the "Requires" field of its Extension Overview. A server that supports the second extension MUST also support the first extension, and MUST list both in its Discovery document. A chain of extensions SHOULD evolve from general to more specific functionality.

8.4. Consumer Implementation

TODO: this section should probably be moved to the rpp-json document

A consumer, which can be a server or client, does not need to generate a distinct set of code classes for every base object and extension combination offered by a particular server. Because extension schemas are additive, self-contained, and independent of one another, a consumer implementation SHOULD instead maintain one set of code classes for each base object and a separate set of code classes for each extension it supports, keeping an extension's classes distinct from the base object's classes rather than flattening their properties together. Composing a base object with whichever extensions apply then becomes a runtime decision, rather than something fixed by code generation for every individual server.

A consumer learns at runtime which extensions a given server supports from the `extensions` list in that server's RPP Discovery response, as defined in [I-D.ietf-rpp-core]. It SHOULD attach, populate, and validate extension class instances only for the extensions the server has advertised as supported, and SHOULD ignore any properties in a response that belong to an extension it does not recognize or does not support.

Because an extension that is not attached to a given base object instance contributes no properties at all, a consumer naturally serializes only the base object together with the properties of whichever extensions are actually in use, without needing to distinguish this case from a property that is merely absent or `null` within a supported extension. Attached extensions are merged flat with the base object on the wire, consistent with the schema composition described in "JSON Schema Composition". Within an attached extension, an optional property without a value continues to follow Rule 2 and is omitted from the output rather than represented as `null`.

8.4.1. Java

For Java clients, a generic library [RPP-JSON-JAVA-LIB] can implement the schema handling described in this document without any code that is specific to a particular extension. The library loads the RPP base schema and the extension schemas supported by a server, uses the `rpp:extends` mappings to compose the effective schema of a base object with `allOf`, applies the `unevaluatedProperties` injection described in "Validation" to the composed schema only, and validates JSON instances against the result. Adding support for a new extension then consists of adding its schema document to the library; no new classes are required.

8.5. Registration

An extension specification that adds properties to an existing object MUST register the added data elements in the RPP Data Object Registry defined in [I-D.ietf-rpp-data-objects], see Section 10. The registration MUST additionally include a dereferenceable URL to the JSON Schema document defining the `$defs` entry described in the Additive Schema Composition section above, and MUST identify the base object type(s) being extended by their registered identifier.

8.6. Validation

A deployment that supports a specific combination of extensions (a profile, as defined in [I-D.ietf-rpp-core]) MUST build, for every extended object type, a combined schema consisting of the base object's definition and the `allOf` branch contributed by every supported extension registered for that object type.

Before using a combined schema to validate a JSON instance, implementations MUST inject `"unevaluatedProperties": false` at every object-schema node of the combined schema, following the algorithm described in [I-D.ietf-rpp-json]. This injection MUST be performed only on the combined, per-deployment schema, and MUST NOT be present in the schema document published by any individual base or extension specification. This ensures undeclared properties are rejected while any supported combination of registered extensions validates successfully.

9. Extension Specification Format

An extension specification SHOULD follow the structure defined in this section, so that all extension specifications are uniform and can be read, reviewed and processed in the same way.

An extension specification contains at least the following sections, in this order:

1. Introduction: the purpose of the extension, the base specifications it builds upon, and its motivation.
2. Extension Overview: the name, the identifier, the version, the RPP version the extension is compatible with, the `$id` of the JSON Schema document, the list of base objects that the extension changes, and the extensions that it requires.
3. Shared Components: the shared components, such as common data types or utility functions, that the extension defines.

4. Data Objects: every data object, component object or process object that the extension introduces, including its object description, data elements and operations.
5. Data Elements: for every base data object, component object or process object that the extension changes, the added data elements and associations. Each added data element MUST be defined using the data element attributes defined in [I-D.ietf-rpp-data-objects] (name, identifier, cardinality, mutability, data type, description and constraints).
6. Operations: every operation that the extension introduces on a new or existing data object.
7. URL Endpoints: the endpoint URLs and HTTP methods that result from the new data objects, associations and operations, as derived by the rules in [I-D.ietf-rpp-core]. This section is informative, an extension MUST NOT define ad-hoc endpoints.
8. Operation Inputs and Outputs: for every existing operation that the extension changes, the added transient and persistent data elements of its input or output.
9. External Data Types: the types, defined in an external specification, that the extension references.
10. Result Codes: the new result codes that the extension defines.
11. Problem Details: the fields that the extension adds to the Problem Detail error object.
12. Query Parameters: the HTTP query parameters that the extension adds to existing operations.
13. HTTP Headers: the HTTP headers that the extension defines.
14. Authentication and Authorization Methods: the authentication and authorization methods that the extension defines.
15. Discovery Document: the properties that the extension adds to the RPP Discovery document, in addition to its entry in the `extensions` list.
16. JSON Schema: the single JSON Schema document of the extension.
17. Examples.
18. IANA Considerations: the registrations that the extension requires, as described in Section 10.
19. Security Considerations.
20. Privacy Considerations.
21. Internationalization Considerations.

All sections MUST all be present. An extension that does not use an extension point MUST state "None" in the corresponding section.

The extension specification MUST define exactly one JSON Schema document, which contains the definitions for all changed and all new objects of the extension. The document MUST have a top-level `$id`, MUST declare the changed base objects using `rpp:extends` as described in "JSON Schema Composition", and MUST NOT contain `unevaluatedProperties` or `additionalProperties` set to `false`, see "Validation". An extension specification MUST NOT split its schema over multiple documents.

The specification MUST include at least one valid JSON example of the request or response representation. Every valid example MUST validate successfully against the effective schema that is composed from the base schema and the JSON Schema document of the extension, as described in Section 8.6.

10. IANA Registries for Extensions

The RPP base specifications define the IANA registries listed below. When an extension specification is published as an RFC, every new entry that the extension requires MUST be registered in the relevant registry, and the IANA Considerations section of the extension specification MUST request these registrations. An extension specification MUST NOT define its own registry for values that belong in one of these registries. If no suitable registry exists for a value that the extension needs to register, the extension specification MUST request the creation of a new registry in its IANA Considerations section, in the RPP registry group defined in [I-D.ietf-rpp-core].

Private extensions, that are not published as an RFC, are not required to register in these registries, but MUST still use identifiers that do not conflict with registered identifiers, as described in [I-D.ietf-rpp-data-objects].

Registry	Defined in	Registration procedure	An extension registers an entry when it
RPP Extensions	[I-D.ietf-rpp-core]	Expert Review	is published; every standardised extension MUST be registered, with its name, version and specification URL
RPP Data Object Registry	[I-D.ietf-rpp-data-objects]	Specification Required	introduces a new data object, component object or process object, adds a data element to an existing object, or defines a new operation or operation parameter
RPP Result Codes	[I-D.ietf-rpp-core]	Expert Review	defines a new result code
RPP Profiles	[I-D.ietf-rpp-core]	Expert Review	defines a standard profile
RPP User Role Values	[I-D.ietf-rpp-data-objects]	Expert Review	defines a new user role
RPP Discovery URLs	[I-D.ietf-rpp-core]	Expert Review	not used by extensions; the registry lists the discovery URL of each RPP server
RPP JSON Schema Extensions	[I-D.ietf-rpp-json] (proposed)	Expert Review	publishes a JSON Schema document for its changed or new objects, registered with the `$id` of the schema
Link Relation Types	[RFC8288], used by [I-D.ietf-rpp-core]	as defined in RFC 8288	defines a new link relation type, in addition to `rpp-process`

Registry	Defined in	Registration procedure	An extension registers an entry when it
Media Types	[RFC6838], used by [I-D.ietf-rpp-core]	as defined in RFC 6838	defines a representation format with its own media type, in addition to `application/rpp+json`
IETF URN Sub-namespace, `urn:ietf:params:rpp`	[I-D.ietf-rpp-core]	as defined in RFC 3553	needs an identifier below the RPP URN namespace, for example `urn:ietf:params:rpp:extension:<name>`

Table 1

The registrations are used as follows:

- The extension MUST be registered in the RPP Extensions registry. The identifier of the extension in the RPP Discovery document is a URN below the RPP URN sub-namespace.
- New and changed objects, data elements, operations and operation parameters MUST be registered in the RPP Data Object Registry. Each entry MUST reference the extension specification, so that implementations can distinguish elements of the base specifications from elements defined by extensions.
- The `@type` value of every new object MUST be the object identifier that is registered in the RPP Data Object Registry, as required by [I-D.ietf-rpp-json].
- New result codes MUST be registered in the RPP Result Codes registry, in the range reserved for extensions.

The RPP data objects also reuse values from registries that are defined outside of RPP, such as the IANA Repository of IDN Practices (IDN tables), the EPP Organization Role Values registry and the EPP Organisation Contact Types registry [RFC8543]. An extension that needs a new value from one of these registries MUST use the procedure of that registry.

TODO: the RPP Extensions registry in [I-D.ietf-rpp-core] and the proposed RPP JSON Schema Extensions registry in [I-D.ietf-rpp-json] overlap, and the RPP Authorization Method registry that is referred to by [I-D.ietf-rpp-json] is not yet defined in [I-D.ietf-rpp-core]. The list of registries in this section must be aligned when these documents are updated.

11. IANA Considerations

TODO

12. Security Considerations

An extension inherits the security services of the protocol structure that it extends. For example, an extension of a data object inherits the authentication, authorisation and confidentiality services that apply to that object. An extension MAY specify additional security services, such as additional authorisation checks. An extension MUST NOT mandate the removal of a security service that is available for the protocol structure that it extends.

An extension is subject to the same security requirements as the base specifications. The Security Considerations section of an extension specification MUST describe the security threats that are introduced by the extension and the measures to counter them, following the guidelines in [RFC3552].

A server MUST apply authentication and authorisation to the data elements, operations and other elements that an extension adds, in the same way as for the elements of the base specifications. A server MUST reject undeclared properties, as described in Section 8.6, and MUST reject the use of extensions that it does not support, as described in the Discovery document extension point.

13. Internationalization Considerations

An extension MUST observe the Internationalization Considerations of the base specifications [I-D.ietf-rpp-core], [I-D.ietf-rpp-data-objects] and [I-D.ietf-rpp-json], and the IETF policy on character sets and languages described in [RFC2277]. Human-readable text that an extension adds to a request or a response MUST be capable of being expressed in the language that is negotiated between the client and the server.

14. Privacy Considerations

TODO

15. Change History

TODO

16. References

16.1. Normative References

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>.

[I-D.ietf-rpp-json] Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC2277] Alvestrand, H., "IETF Policy on Character Sets and Languages", BCP 18, RFC 2277, DOI 10.17487/RFC2277, January 1998, <https://www.rfc-editor.org/info/rfc2277>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/info/rfc6838>.

[RFC8288] Nottingham, M., "Web Linking", RFC 8288, DOI 10.17487/RFC8288, October 2017, <https://www.rfc-editor.org/info/rfc8288>.

[RFC8543] Zhou, L., Kong, N., Yao, J., Gould, J., and G. Zhou, "Extensible Provisioning Protocol (EPP) Organization Mapping", RFC 8543, DOI 10.17487/RFC8543, March 2019, <https://www.rfc-editor.org/info/rfc8543>.

16.2. Informative References

[I-D.wullink-rpp-extension-guidelines] Wullink, M. and P. Kowalik, "Guidelines for Extending the RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-wullink-rpp-extension-guidelines-00, 2026, <https://datatracker.ietf.org/doc/html/draft-wullink-rpp-extension-guidelines-00>.

[JSON-SCHEMA] JSON Schema, "JSON Schema: A vocabulary for describing JSON Data", 2020, <https://json-schema.org/draft/2020-12/json-schema-core>.

[RFC3552] Rescorla, E. and B. Korver, "Guidelines for Writing RFC Text on Security Considerations", BCP 72, RFC 3552, DOI 10.17487/RFC3552, July 2003, <https://www.rfc-editor.org/info/rfc3552>.

[RPP-JSON-JAVA-LIB] SIDN Labs, "rpp-json-java-lib: Java library for RPP JSON Schema extensions", 2026, <https://github.com/SIDN/rpp-json-java-lib>.

Appendix A. Example Extension

This appendix is informative. It shows an extension specification that follows the structure defined in "Extension Specification Format", and it shows how the extension can be validated with a generic Java library. The example extension adds the property `extProperty1` to the Domain Name data object.

This example extension, its schema and the examples above can be validated with the generic Java library in the rpp-json-java-lib repository [RPP-JSON-JAVA-LIB]. The library loads the base schema and the extension schemas supported by a deployment, builds effective schemas from the `rpp:extends` mappings, applies the injection of `unevaluatedProperties` described in Section 8.6, and validates JSON instances against the result. The repository contains this example extension, the example instances and unit tests, and its README describes how to build and run them.

A.1. Introduction

This document provides an example of an extension specification that follows the guidelines defined in [I-D.wullink-rpp-extension-guidelines]. The example demonstrates how to add a new property to an existing data object and how to define the corresponding JSON Schema and examples.

A.2. Extension Overview

Field	Value
Name	RPP Example Extension Property
Identifier	urn:ietf:params:rpp:extension:example-ext-property
Version	1.0
RPP version	1.0
Schema `$id`	https://rpp.example/schemas/ext-domain-extproperty1.json
Changed base objects	domainName
New objects	None
Requires	None

Table 2

A.3. Shared Components

None.

A.4. Data Objects

None.

A.5. Data Elements

A.5.1. Domain Name Data Object

- Identifier: domainName

The following data element is added to the Domain Name Data Object.

- Extension Property 1
  - Identifier: extProperty1
  - Cardinality: 0-1
  - Mutability: create-only
  - Data Type: String

- Description: An example property that is added to the Domain Name Data Object by the extension.
- Constraints:
  - The value MUST contain between 1 and 64 characters.
  - The value MUST be provided in the Create operation.

The Read operation returns `extProperty1` when the domain name was created with a value for it. No other operations are changed.

A.6. Operations

None.

A.7. URL Endpoints

None. The extension adds no data object, association or operation, so no endpoint is added.

A.8. Operation Inputs and Outputs

None.

A.9. External Data Types

None.

A.10. Result Codes

None.

A.11. Problem Details

None.

A.12. Query Parameters

None.

A.13. HTTP Headers

None.

A.14. Authentication and Authorization Methods

None.

A.15. Discovery Document

None.

A.16. JSON Schema

The extension defines a single JSON Schema document. The document declares, using `rpp:extends`, that it extends the create and read definitions of the base object, and contributes the new property in a separate definition:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/schemas/ext-domain-extproperty1.json",

  "rpp:extends": {
    "https://rpp.example/rpp/schema.json#/$defs/domainObject.create": "#/
$defs/ext.domain.create",
    "https://rpp.example/rpp/schema.json#/$defs/domainObject.read": "#/$defs/
ext.domain.read"
  },

  "$defs": {
    "ext.domain.create": {
      "type": "object",
      "properties": {
        "extProperty1": { "type": "string", "minLength": 1, "maxLength": 64 }
      },
      "required": ["extProperty1"]
    },
    "ext.domain.read": {
      "type": "object",
      "properties": {
        "extProperty1": { "type": "string" }
      }
    }
  }
}
```

A.17. Examples

The following instance is valid, because `extProperty1` is declared by the supported extension:

```
{
  "@type": "domainName",
  "name": "example.example",
  "registrant": { "@type": "contact", "id": "c-1234" },
  "nameservers": [
    { "@type": "host", "hostName": "ns1.example.net" }
  ],
  "extProperty1": "some value"
}
```

The following instances are invalid. The first omits the required `extProperty1`, the second gives `extProperty1` the wrong type, and the third contains `extProperty2`, which no supported extension declares and which is therefore rejected by the injected `unevaluatedProperties` keyword:

```
{ "@type": "domainName", "name": "example.example" }
```

```
{ "@type": "domainName", "name": "example.example", "extProperty1": 42 }
```

```
{
  "@type": "domainName",
  "name": "example.example",
  "extProperty1": "some value",
  "extProperty2": "not declared by any supported extension"
}
```

The same base schema and the same instance that uses `extProperty1` are rejected by a deployment that does not support the extension, since in that deployment `extProperty1` is an undeclared property. This is the behavior required by "Validation": the set of accepted properties is determined only by the extensions that the deployment supports.

A.18. Composition and Validation

For a deployment that supports the example extension, the effective schema for `domainObject.create` references the base definition and the extension definition, and has its own `$id`:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://rpp.example/rpp/effective/domain.create.json",
  "$ref": "#/$defs/effective.domainObject.create",
  "$defs": {
    "effective.domainObject.create": {
      "allOf": [
        { "$ref": "https://rpp.example/rpp/schema.json#/$defs/
domainObject.create" },
        { "$ref": "https://rpp.example/schemas/ext-domain-extproperty1.json#/
$defs/ext.domain.create" }
      ]
    }
  }
}
```

A.19. IANA Considerations

TODO:

A.20. Security Considerations

TODO:

A.21. Privacy Considerations

TODO:

A.22. Internationalization Considerations

TODO:

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/